MASTER DEVELOPMENT PROMPT: KaamConnect – All Services, One App

1. Project Overview

Build a complete, modern, responsive, database-connected service marketplace named KaamConnect – All Services, One App.

The platform will connect customers with verified local service professionals across India. Customers should be able to find, compare, book, contact, and review workers for household services, repairs, maintenance, personal care, and other local jobs.

This project must be a functional application, not merely a UI mockup, landing page, or collection of static screens.

Mandatory Technology Stack

Use the following technologies:

- Frontend: HTML5, CSS3, vanilla JavaScript (ES6+).
- Backend: Python with Flask.
- Database: MySQL 8.0+.
- Database connectivity: SQLAlchemy ORM or a reliable MySQL connector.
- API architecture: RESTful APIs returning JSON where appropriate.
- Authentication: Secure session-based authentication with role-based access control.
- Charts: Chart.js for admin analytics.
- Icons: Font Awesome or Lucide icons.
- Optional installability: Progressive Web App (PWA) support for compatible mobile devices.

Do not replace the required stack with React, Node.js, PHP, Firebase, MongoDB, or another framework or database.

Keep the frontend, backend, database models, API routes, and templates modular and maintainable.

2. Application Goals

Create a professional Indian home-services marketplace with:

- A customer-facing website.
- A worker registration portal and dashboard.
- An administrator dashboard.
- A MySQL database for persistent application data.
- A complete demo booking and payment workflow.
- Responsive layouts for mobile phones, tablets, and desktop computers.
- English and Hindi language options.
- Realistic sample data suitable for a school science exhibition.

Make the website installable as a PWA where supported. Clearly distinguish an installable web app from a native Android application; do not claim native Android functionality unless it is actually implemented.

3. UI and Visual Design

Use a professional blue, white, and orange color palette.

Suggested design tokens:

- Primary blue: #1769E0
- Dark blue: #123B72
- Accent orange: #FF8A24
- Background: #F5F8FC
- White cards with subtle shadows and rounded corners.
- Accessible text colors and visible focus states.

Design requirements:

- Attractive splash screen with a KaamConnect logo.
- Responsive navigation and mobile menu.
- Home screen with search, service categories, popular workers, promotional banners, and nearby services.
- Worker profile cards with photographs, ratings, experience, pricing, and availability.
- Consistent buttons, forms, modals, alerts, loading indicators, and empty states.
- Mobile-first layouts and touch-friendly controls.
- Clear booking progress and confirmation messages.
- Responsive admin dashboard with charts and summary cards.
- Helpful error pages and a custom 404 page.

Use realistic sample worker photographs or appropriately licensed placeholder images. Never use photographs to imply that a fictional worker is a verified real person.

4. User Roles and Permissions

Implement three primary roles.

Customer

- Register, log in, log out, and reset passwords.
- Manage profile details and saved addresses.
- Search for workers and services.
- Create, view, reschedule where permitted, and cancel bookings.
- Pay through demo payment options.
- Contact workers using protected communication features.
- Submit reviews after completed jobs.
- Report problems and contact support.

Worker

- Register and submit professional information.
- Upload a profile photograph and relevant certificates.
- Select services, skills, prices, service areas, and working hours.
- Submit identity verification information through a secure process.
- Wait for administrator approval before becoming publicly bookable.
- Accept or reject booking requests.
- Update job progress according to permitted transitions.
- View earnings, completed jobs, booking history, and payment records.
- Manage availability and receive notifications.

Administrator

- Use a protected administrator login.
- View and manage customers, workers, services, and bookings.
- Approve, reject, suspend, or reactivate worker profiles.
- Manage service categories and pricing rules.
- Review complaints, reports, and inappropriate reviews.
- Monitor demo transactions and platform fees.
- View analytics and operational summaries.
- Review audit logs and authorized administrative actions.

Enforce permissions on the server for every protected route and API. Hiding buttons in the browser is not sufficient security.

5. Service Categories

Create editable categories for:

1. Maid and House Cleaning
2. Cook and Kitchen Help
3. Babysitter and Caregiver
4. Electrician
5. Plumber
6. AC and Refrigerator Technician
7. Carpenter
8. Painter
9. Washing Machine Repair
10. Mobile and Computer Repair
11. Driver
12. Gardener
13. Mechanic
14. Pest Control
15. Packers and Movers
16. Beauty and Personal Care
17. Construction Workers
18. Delivery and Other Local Services

Each category must support:

- Category name and description.
- Icon and optional image.
- Active/inactive status.
- Related services and optional subcategories.
- Configurable pricing type.
- Admin create, edit, and deactivate operations.

Do not hard-code categories throughout the application. Load them from MySQL.

6. Customer Features

Registration and Login

- Name, phone number, email where applicable, and password.
- Server-side validation and duplicate-account prevention.
- Secure password hashing.
- Login, logout, session expiration, and password-reset workflow.
- Demo authentication must be clearly identified where external delivery services are unavailable.

Customer Profile

- Personal information.
- Saved addresses with labels such as Home and Work.
- Booking history and upcoming bookings.
- Notification preferences.
- Account settings and logout.

Search and Discovery

- Search by worker name, category, service, skill, and location.
- Filter by price, rating, experience, availability, and verified status.
- Sort by relevance, price, rating, or experience.
- Paginated results.
- Display useful messages when no results are found.
- Only show active, approved, bookable workers in public search.

If real geolocation is unavailable, provide a manual city, area, or PIN code selection. Never claim that sample data represents live GPS-based worker availability.

Worker Profiles

Display:

- Profile photograph.
- Name and verification status.
- Services and skills.
- Experience.
- Service area.
- Ratings and customer reviews.
- Pricing model and estimated charges.

- Working hours and available slots.
- Booking button and protected contact option.

7. Complete Booking Workflow

Implement the following end-to-end workflow:

1. Customer selects a service category.
2. Customer searches and chooses an eligible worker.
3. Customer opens the worker profile.
4. Customer selects a service and one-time or recurring option where supported.
5. Customer selects an available date and time.
6. Customer selects or enters a service address.
7. Customer provides the job description and optional permitted attachments.
8. Backend validates the worker's approval, service area, availability, and pricing.
9. Application calculates and displays an itemized estimate.
10. Customer reviews the cancellation policy and confirms the booking.
11. Backend creates a unique booking ID and persists the booking in MySQL.
12. Worker receives a booking request and accepts or rejects it.
13. Customer receives booking status updates.
14. Worker updates the job through the permitted status sequence.
15. Booking is completed, the final bill is recorded, and the customer can submit a review.

Booking Statuses

Implement these statuses:

- Pending
- Accepted
- On the Way
- In Progress
- Completed
- Cancelled
- Rejected

Define permitted status transitions and enforce them server-side. Record who made each change and when.

Booking Reliability

- Generate unique, non-guessable public booking references.
- Prevent overlapping confirmed reservations for the same worker.
- Recheck availability when confirming the booking, not only when displaying available slots.
- Use database transactions and appropriate locking or constraints to handle simultaneous booking attempts.
- Prevent duplicate submissions and duplicate charges through idempotency mechanisms where applicable.
- Store booking price snapshots so that future price changes do not rewrite historical bills.
- Implement cancellation rules and record cancellation reasons.
- Support recurring bookings only when scheduling, availability, and cancellation rules are implemented.

8. Pricing and Payments

Create a transparent itemized bill containing:

- Base service charge.
- Additional service charges where applicable.
- Platform fee.
- Discount.
- Applicable taxes, only when configured.
- Final estimated or payable total.

Use a centralized backend pricing service. Never trust totals calculated only by JavaScript.

Payment Methods

1. Pay After Service / Cash on Delivery.
2. Simulated online payment for demonstrations.

For simulated payments:

- Display a clear “DEMO TRANSACTION — NO REAL MONEY” notice.
- Provide success, failure, and cancellation scenarios.
- Save transaction status, amount, method, and reference in MySQL.
- Never collect or store actual card numbers, CVVs, or banking credentials.

- Do not mark a real payment as successful without an authorized payment-provider confirmation.

Include payment statuses such as Pending, Successful, Failed, Refunded, and Not Required where appropriate.

Real online payments must remain disabled until a legitimate payment gateway, credentials, webhook verification, and server-side verification are configured.

9. Worker Registration and Dashboard

Build a complete worker registration process with:

- Full name and contact details.
- Service category and skills.
- Experience and description.
- Service area and PIN codes.
- Pricing and pricing type.
- Working hours and weekly availability.
- Profile photograph.
- Relevant certificates and verification documents.

Approval Process

Use states such as:

- Draft
- Submitted
- Under Review
- Approved
- Rejected
- Suspended

Only approved and active workers can receive public bookings.

Protect identity documents from public access. Store uploads securely, validate file type and size, use randomized filenames, and restrict access to authorized personnel.

Do not display sensitive identity numbers or documents in public profiles.

Worker Dashboard

Include:

- Today's jobs.
- Pending booking requests.
- Accepted and upcoming bookings.
- Completed and cancelled jobs.
- Earnings summary.
- Booking history.
- Availability controls.
- Profile completeness and verification status.
- Notifications and support access.

Distinguish gross service value, platform fees, adjustments, and net demo earnings.

10. Contact and Notifications

Implement an in-app booking conversation feature or a protected contact mechanism.

- Customers and workers may access conversations only for authorized bookings.
- Validate and authorize every message.
- Store messages in MySQL.
- Display timestamps and unread indicators where implemented.
- Avoid exposing personal phone numbers publicly.
- Add in-app notifications for booking creation, acceptance, rejection, cancellation, and completion.

If email, SMS, WhatsApp, or push notification providers are not configured, provide a working in-app notification system and clearly label external notifications as requiring configuration.

11. Reviews, Complaints, and Support

Reviews

- Ratings from one to five stars.

- Optional written feedback.
- Only customers with completed bookings can review the relevant worker.
- Prevent duplicate reviews for the same booking unless editing is explicitly supported.
- Allow administrators to moderate reported or inappropriate reviews.
- Calculate displayed ratings from actual stored reviews.

Complaints

- Complaint categories and descriptions.
- Booking reference where applicable.
- Complaint status and administrative notes.
- Submission date and resolution history.
- Restricted access to complaint details.

Help and Support

- FAQs.
- Booking and payment guidance.
- Cancellation policy.
- Contact/support form.
- Support ticket history and status.

12. Administrator Dashboard

Create an administrator dashboard with:

- Total registered customers.
- Total registered workers.
- Approved, pending, and suspended workers.
- Total bookings.
- Active bookings.
- Completed and cancelled bookings.
- Demo payment totals and demo platform revenue.
- Category-wise service demand.
- Booking trends over time.
- Worker approval queue.
- Complaints and reported reviews.
- Recent activity and audit history.

Use Chart.js for real charts driven by MySQL data.

All statistics must be calculated from database records. Label sample-data statistics as demo data. Never present simulated transactions as actual company revenue.

13. Required Screens

Implement all screens with working navigation:

1. Splash screen.
2. Welcome and onboarding.
3. Login and registration.
4. Customer home.
5. Service categories.
6. Worker search and filters.
7. Worker profile and details.
8. Booking form.
9. Booking confirmation.
10. My Bookings.
11. Customer profile and addresses.
12. Worker registration.
13. Worker dashboard.
14. Worker booking requests.
15. Worker earnings.
16. Administrator dashboard.
17. Help and support.

Also implement the necessary supporting screens:

- Forgot/reset password.
- Worker verification status.
- Booking details and status timeline.
- Payment summary and demo checkout.
- Notifications.
- Chat or protected contact.
- Reviews and complaints.
- Category management.
- User and worker management.
- Error, loading, empty, and unauthorized states.

14. MySQL Database Design

Create a normalized relational database with suitable primary keys, foreign keys, unique constraints, indexes, timestamps, and data types.

Include tables for:

- users
- customer_profiles
- customer_addresses
- worker_profiles
- worker_documents
- service_categories
- services
- worker_services
- worker_service_areas
- worker_availability
- bookings
- booking_status_history
- booking_items
- recurring_booking_rules, if recurring bookings are implemented
- reviews
- payments
- conversations
- messages
- notifications
- support_tickets
- complaints
- coupons or discounts, if implemented
- platform_settings
- admin_audit_logs

Adjust the schema when necessary to avoid unnecessary duplication.

Important rules:

- Store passwords as secure hashes, never plaintext.
- Use foreign keys and appropriate deletion policies.
- Use decimal types for monetary values.
- Store timestamps consistently and display them in the configured local timezone.
- Add indexes for searches, booking lookups, statuses, and foreign keys.

- Use transactions for multi-step booking and payment-record operations.
- Never delete financial or booking history silently.
- Use migrations or versioned SQL scripts to manage schema changes.

Provide an ER diagram or Mermaid entity-relationship diagram and explain important table relationships.

15. Python Flask Backend

Organize the backend into maintainable modules, for example:

- app initialization and configuration
- database models
- authentication and authorization
- customer routes
- worker routes
- booking routes
- service/category routes
- payment routes
- review routes
- messaging and notifications
- administrator routes
- validation and pricing services
- error handling
- database migrations and seed utilities

Use Flask blueprints or an equivalent modular routing pattern.

Implement RESTful endpoints for:

- Authentication.
- Customer profiles and addresses.
- Categories and services.
- Worker search and profiles.
- Worker registration and verification.
- Availability checks.
- Booking creation, retrieval, cancellation, and status updates.
- Payment demo flows.
- Reviews.
- Messages and notifications.
- Support tickets and complaints.

- Administrator management and analytics.

Provide consistent JSON response formats, appropriate HTTP status codes, input validation, pagination, and useful error messages.

Do not expose database passwords, secret keys, stack traces, or internal error details in production responses.

16. Security and Privacy

Implement security throughout the application:

- Hash passwords using Werkzeug's secure password-hashing utilities.
- Use server-side role and resource authorization.
- Use parameterized database queries or a properly configured ORM.
- Protect state-changing forms and cookie-authenticated endpoints against CSRF.
- Validate and normalize user inputs on the server.
- Escape untrusted output to reduce XSS risks.
- Configure secure session cookies and session expiration.
- Use rate limiting for login and sensitive endpoints where feasible.
- Apply safe upload restrictions and prevent path traversal.
- Restrict CORS if cross-origin access is required.
- Use environment variables for secrets and database credentials.
- Use HTTPS in production.
- Protect personal data and limit collection to what is necessary.
- Record important administrative actions.
- Never expose another user's private bookings, messages, or documents.
- Do not ship an undocumented universal admin password or production backdoor.

Create an initial administrator through a documented, secure setup command or environment-configured bootstrap process. Do not expose administrator credentials in public source code.

17. Frontend Architecture

Use reusable HTML templates and shared CSS/JavaScript modules.

Suggested structure:

- Shared header and navigation.
- Reusable worker cards.
- Reusable service-category cards.
- Form components and validation helpers.
- Booking status badges and timelines.
- Modal dialogs and toast notifications.
- Shared API client.
- Customer, worker, and administrator layouts.

Use semantic HTML, responsive CSS, accessible labels, keyboard navigation, and sufficient color contrast.

Implement English and Hindi using centralized translation dictionaries and a persistent language preference. Ensure the important buttons, forms, validation messages, booking statuses, and navigation labels are translated.

Avoid excessive inline JavaScript, duplicate code, hard-coded user data, and direct database access from the browser.

18. Demo Data

Create a database seeding script with realistic fictional Indian sample data.

Include:

- Multiple customers.
- Workers from different categories.
- Different service prices and experience levels.
- Different ratings and reviews.
- Approved, pending, and rejected worker applications.
- Available and unavailable time slots.
- Bookings in different statuses.
- Simulated payments.
- Support tickets and notifications.

Include examples from Indian cities and neighborhoods without claiming the sample workers are real or actually available.

Provide clearly documented demo accounts for each role through a local-only seed/setup process. Never enable publicly accessible default credentials in production.

19. Validation and Testing

Add automated tests for the critical workflows.

Test at minimum:

- Registration and login.
- Invalid inputs and duplicate accounts.
- Role-based access control.
- Worker approval restrictions.
- Search and filtering.
- Booking creation and price calculation.
- Overlapping booking prevention.
- Simultaneous booking attempts.
- Cancellation rules.
- Valid and invalid booking status transitions.
- Demo payment success and failure.
- Review eligibility.
- Administrator-only actions.
- Private document and message access.

Use pytest and a separate test database. Provide instructions for running tests and explain any remaining manual testing.

Do not claim a test passes unless it has actually been run.

20. Project Structur